



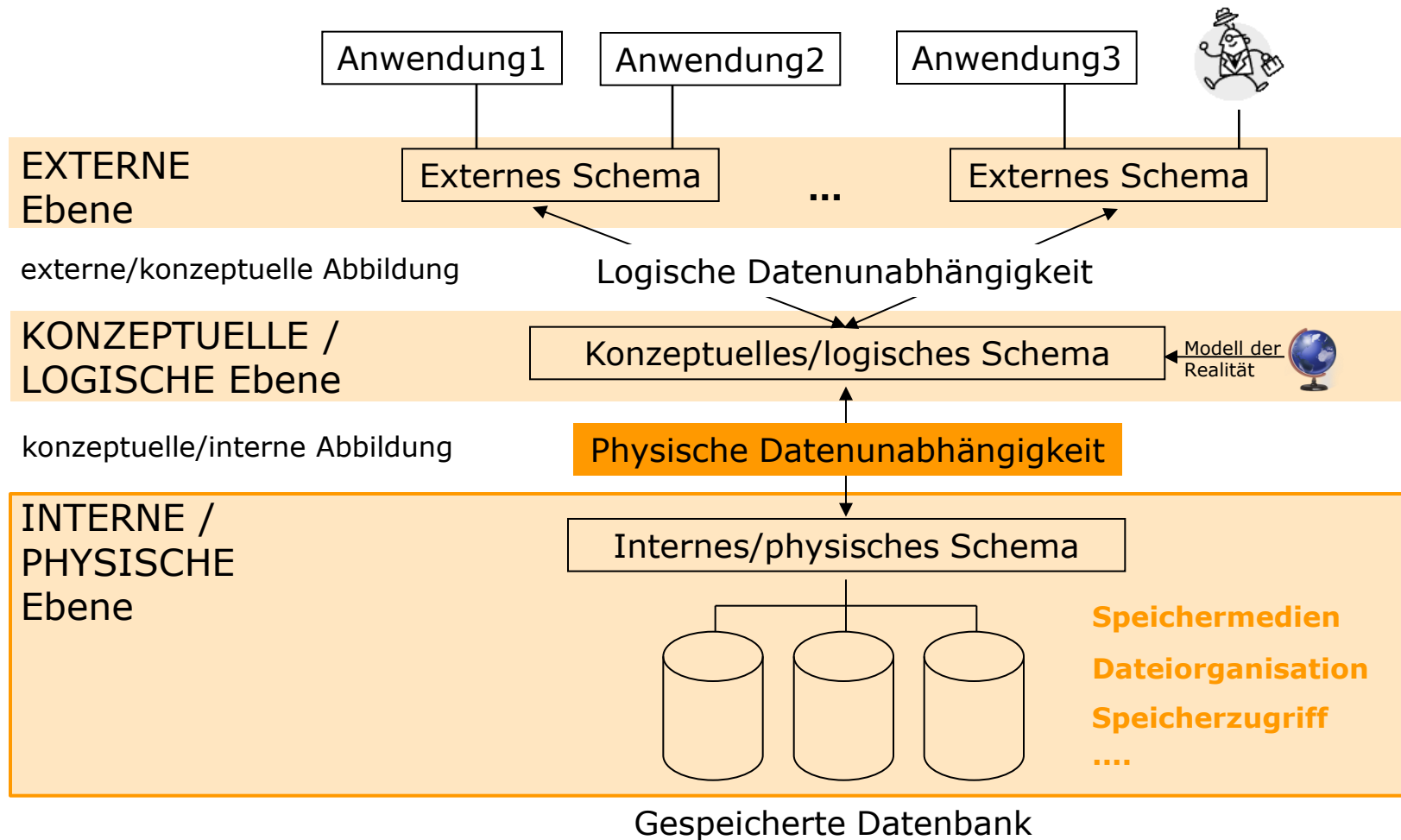
Winand, M.: Use the Index, Luke! [Anleitung zur SQL-Performance für Entwickler](#)

Überblick

- Internes Schema
- Abbildung konzeptuelle auf interne Ebene
- Dateiorganisation und Zugriffspfade
- Indexierung
- Indexstrukturen in SQL (ORACLE)

Motivation

ANSI/X3/SPARC: Normungsvorschlag
für Systemarchitektur von DBMS



- Änderungen an der physischen Speicher- oder der Zugriffsstruktur (beispielsweise das Anlegen oder Entfernen einer Indexstruktur) hat keine Auswirkungen auf das logische/konzeptuelle Schema.

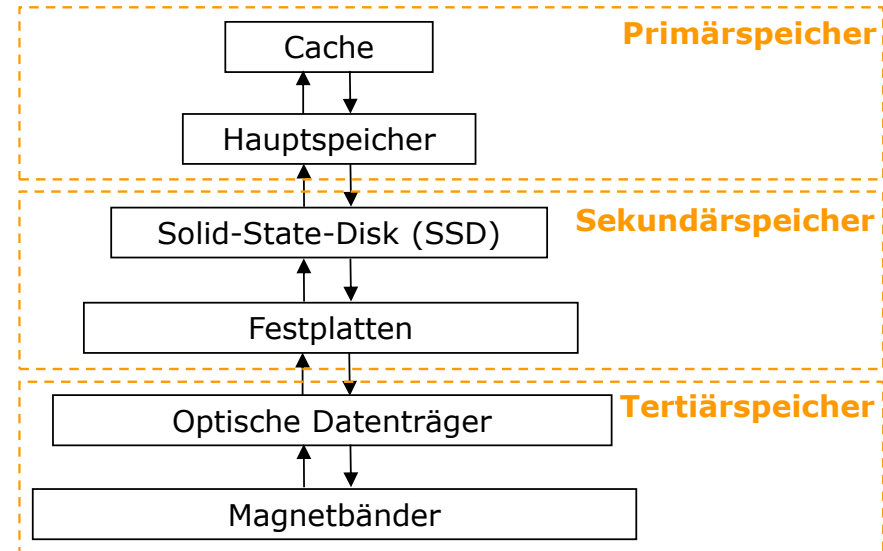
Speichermedien

- **Verschiedene Arten** von Speichermedien sind für Datenbanksysteme relevant.
- Speichermedien lassen sich in **Speicherhierarchie** anordnen.
- **Klassifizierung** der Speichermedien nach:
 - Zugriffsgeschwindigkeit
 - Kosten pro Dateneinheit
 - Verlässlichkeit
 - Datenverlust durch Stromausfall oder Systemabsturz
 - Physische Fehler des Speichermediums
 - Flüchtige vs. persistente Speicher
 - Flüchtig (volatile): Inhalt geht nach Ausschalten verloren
 - Persistent (non-volatile): Inhalt bleibt auch nach Ausschalten

Speicherhierarchie

- Speichermedien können hierarchisch nach **Geschwindigkeit** und **Kosten** geordnet werden:

- **Primärspeicher**: flüchtig, schnell, sehr teuer
 - Cache, Hauptspeicher
- **Sekundärspeicher**: persistent, langsamer, günstiger
 - Magnetplatten, Flash Speicher auch Online-Speicher genannt
- **Tertiärspeicher**: persistent, sehr langsam, sehr günstig
 - Magnetbänder, optischer Speicher auch Offline-Speicher genannt

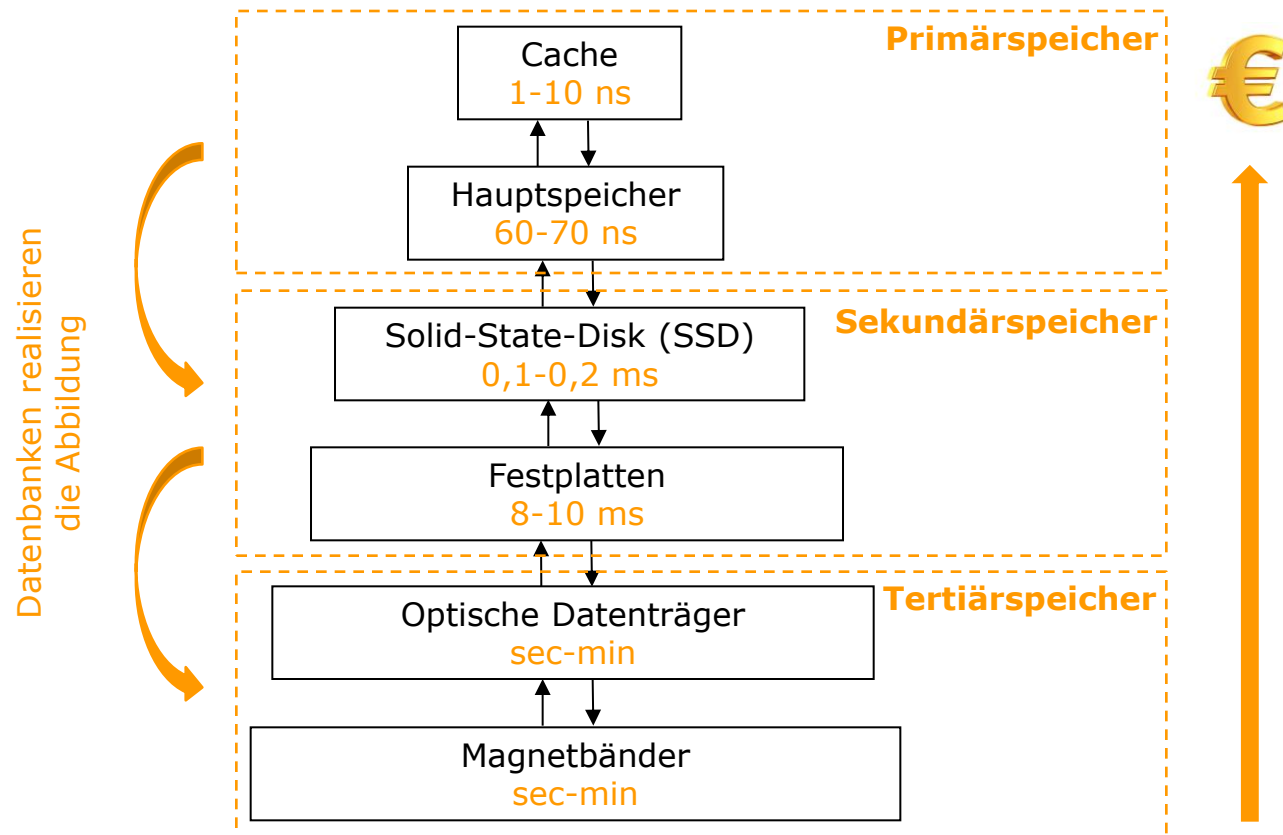


- Datenbank muss mit Speichermedien auf allen Ebenen umgehen

Speicherhierarchie

Zugriffszeiten

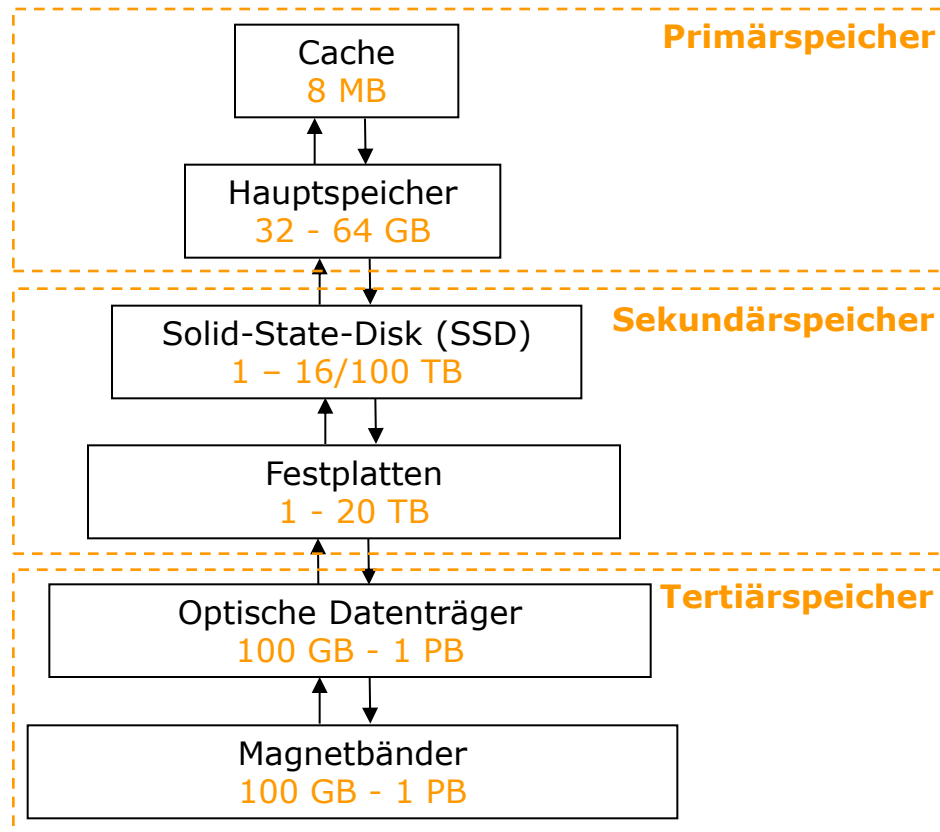
- Speichermedien nach **Zugriffsgeschwindigkeit** geordnet:



Speicherhierarchie

Kapazitäten

- Speichermedien nach **Speicherkapazität** geordnet:



Speicherzugriff

- **Puffer:** Hauptspeicherbereich für Kopien von Platten-Blöcken
- **Puffer Manager:** Subsystem zur Verwaltung des Puffers
 - Anzahl der Plattenzugriffe soll minimiert werden
 - ähnlich der virtuellen Speicherverwaltung in Betriebssystemen

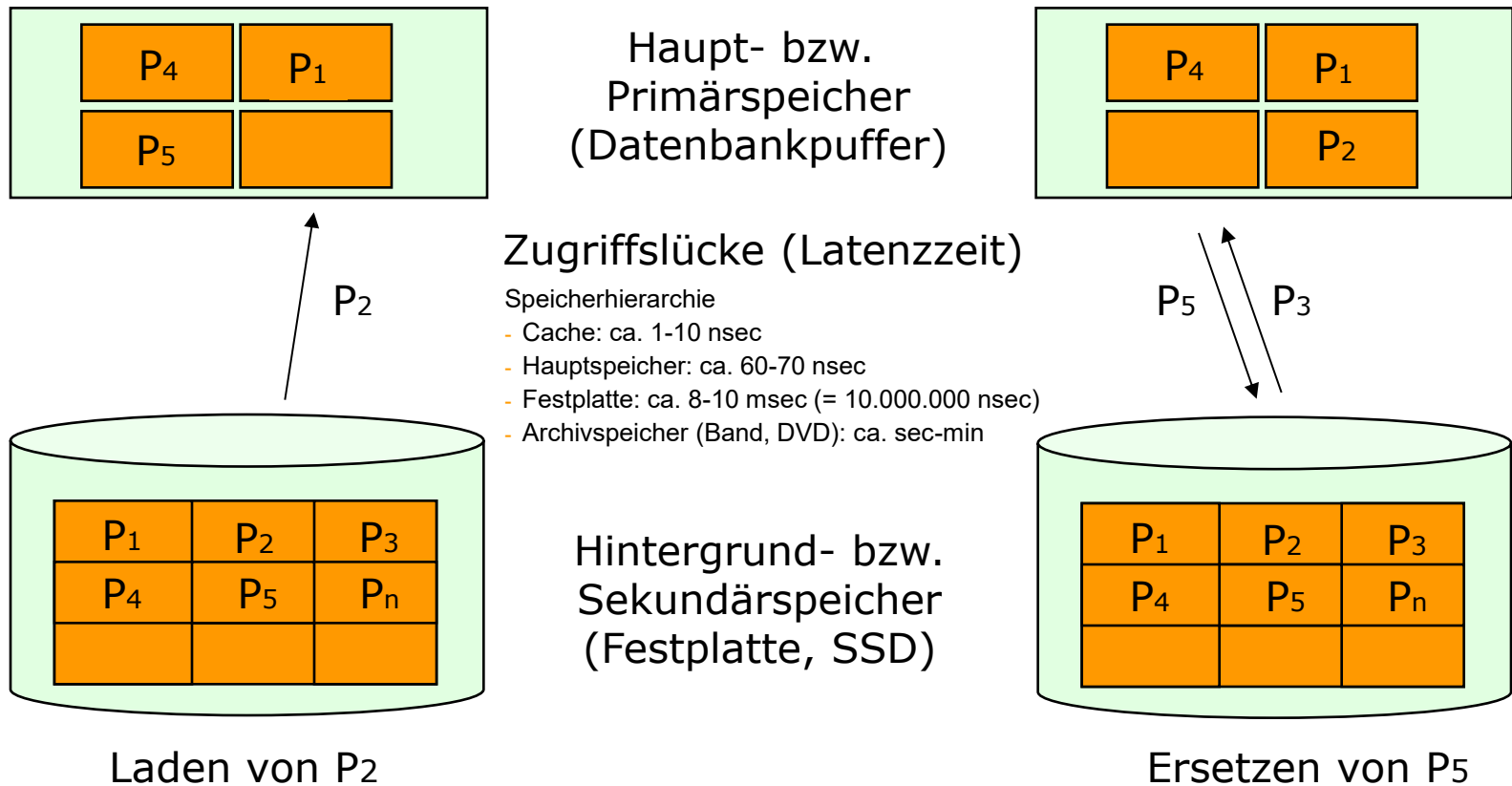


Abbildung konzeptuelle/logische Ebene auf interne Ebene

Konzeptuelle Ebene		Interne Ebene		Dateisystem/Platte
Relationen	→	Logische Dateien	→	Physische Dateien
Tupel	→	Datensätze (Records)	→	Seiten/Blöcke
Attributwerte	→	Felder	→	Bytes

- Um Operationen (Anfragen, Hinzufügen, Löschen, Ändern) effizient durchführen zu können, setzt die interne Ebene des Datenbanksystems geeignete **Dateiorganisation** und **Zugriffsstrukturen** ein.
- Anwendungen sind von Änderungen des internen Schemas nicht betroffen (= physische Datenunabhängigkeit)
- Abbildungen in beiden Richtungen sind durch Metadaten beschrieben, die im **Data Dictionary** verwaltet werden.

Dateiorganisation

Anordnung der Datensätze in Dateien

- Speicherungsform der Datensätze in der Datendatei bestimmt die Effizienz der Zugriffs- und Modifikationsoperationen auf den Datensätzen.
 - **unsortierte Speicherung**: Heap-Organisation
 - **sortierte Speicherung**: sequenzielle Organisation
 - **gestreute Speicherung**: Hash-Organisation

Zugriffsstrukturen

- Zugriffspfad beschleunigt Datenzugriff in der Datendatei, bspw. durch Index bzw. Indexdatei
 - **Indexdatei** hat Einträge der Form (↓Suchschlüssel, ↑Zeiger):
 - ↓Suchschlüssel: Attribut(menge) nach der Daten gesucht werden
 - ↑Zeiger: Verweis auf Datensatz (TID - Tuple Identifier)
 - **Suchschlüssel** dürfen doppelt vorkommen (im Gegensatz zu Primärschlüssel)
 - Wichtigste Zugriffsarten sind
 - **Punktanfrage**: bspw. Kunde mit **KNr=33**
 - **Mehrpunktanfrage**: bspw. Kunden mit **ort=Linz**
 - **Bereichsanfrage**: bspw. Kunden, mit **Umsatz > 100.000**
- Indexdatei kann wiederum in einer bestimmten Dateiorganisationsform gespeichert sein

Abbildung Relationen auf Sekundärspeicher

Mögliche Datenorganisation mit Zugriffsstrukturen

Eine Relation

Zugriffsstrukturen

- B-Tree
- B+-Tree
- B*-Tree
- ...

Dateiorganisation:

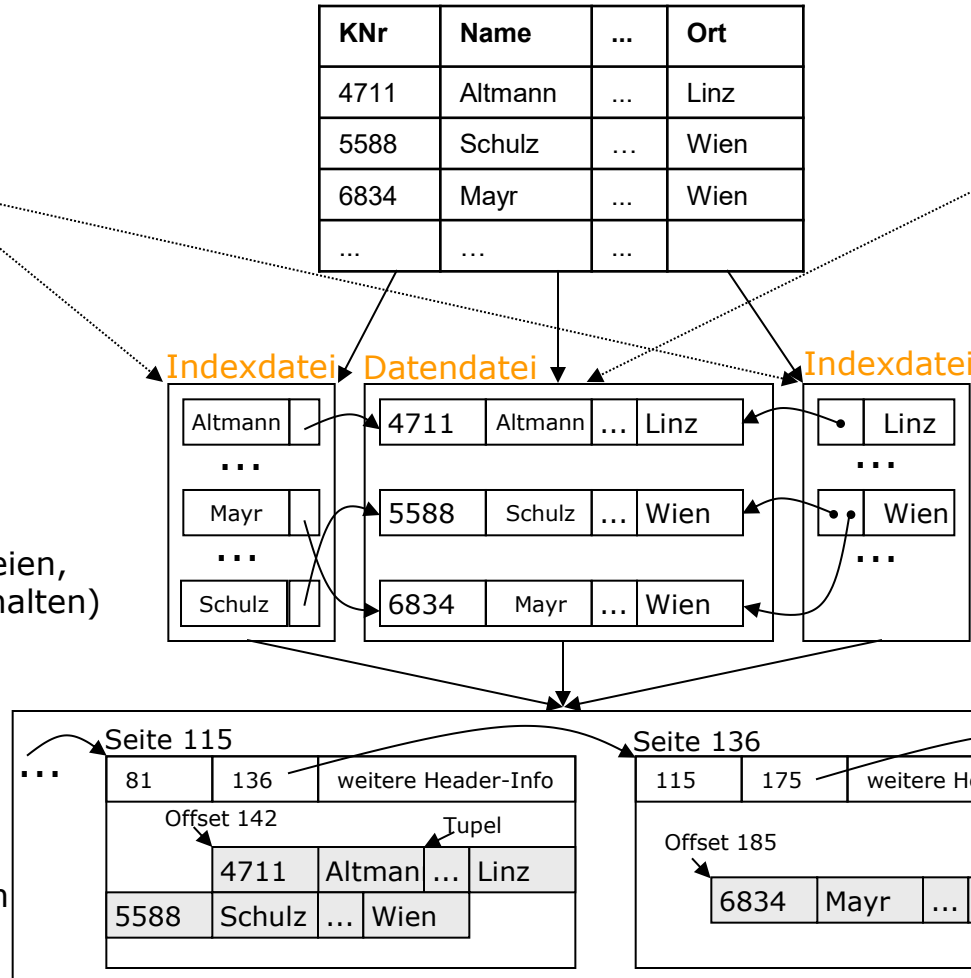
- unsortierte Speicherung
- sortierte Speicherung
- gestreute Speicherung (Hash)
- ...

wird in mehrere
logische Dateien
abgebildet

(Datendatei und Indexdateien,
die Zugriffsstrukturen enthalten)

und in einer
physischen Datei gespeichert.

(Nutzdaten und Indexdaten
werden meist physisch
getrennt, bspw. separate
Tablespaces in Oracle)

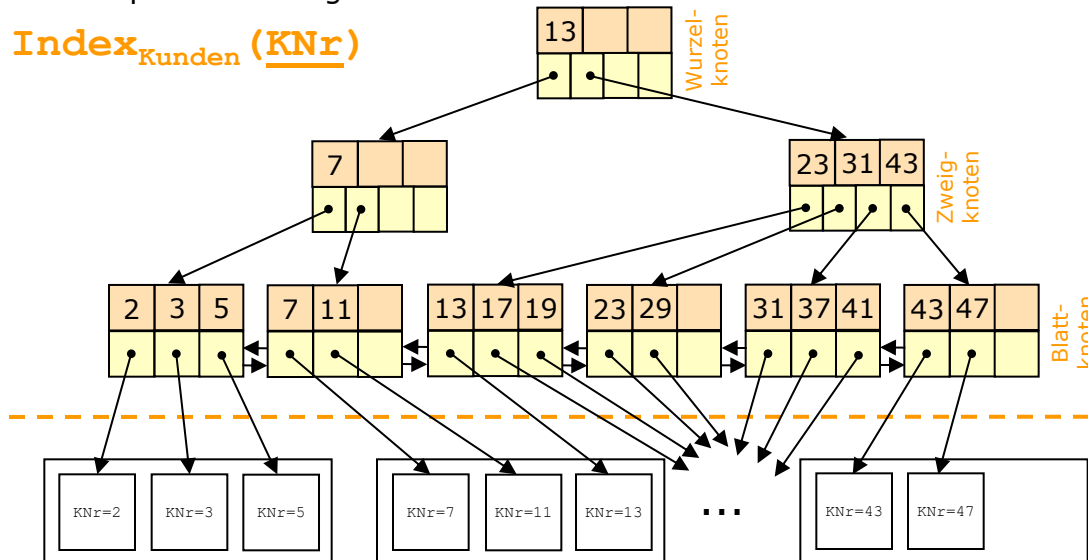


Blocken
Pufferverwaltung
Seitenersetzung
Fragmentierung
...
~
Betriebssystem
(Cooked vs.
Raw Devices)

Indexstrukturen

Primärindex

- Datendatei
 - sequentiell nach Suchschlüssel geordnet
 - Suchschlüssel ist eindeutig
 - d.h., zu jedem Wert eines Suchschlüssels gibt es nur einen Datensatz in der Datendatei - trifft bei Primärschlüssel oder Schlüsselkandidaten zu
- Indexdatei
 - sequentiell nach Suchschlüssel geordnet
 - Primärindex garantiert, dass keine Duplikate enthalten sind
- Effiziente Zugriffsarten:
 - Punkt- und Bereichsanfragen
 - nicht-sequentieller Zugriff (random access)
 - sequentieller Zugriff nach Suchschlüssel sortiert



Indexdatei

Mehrstufiger Index
(Dateiorganisation als
balancierter B⁺-Suchbaum)

Dicht besetzter Index

Datendatei

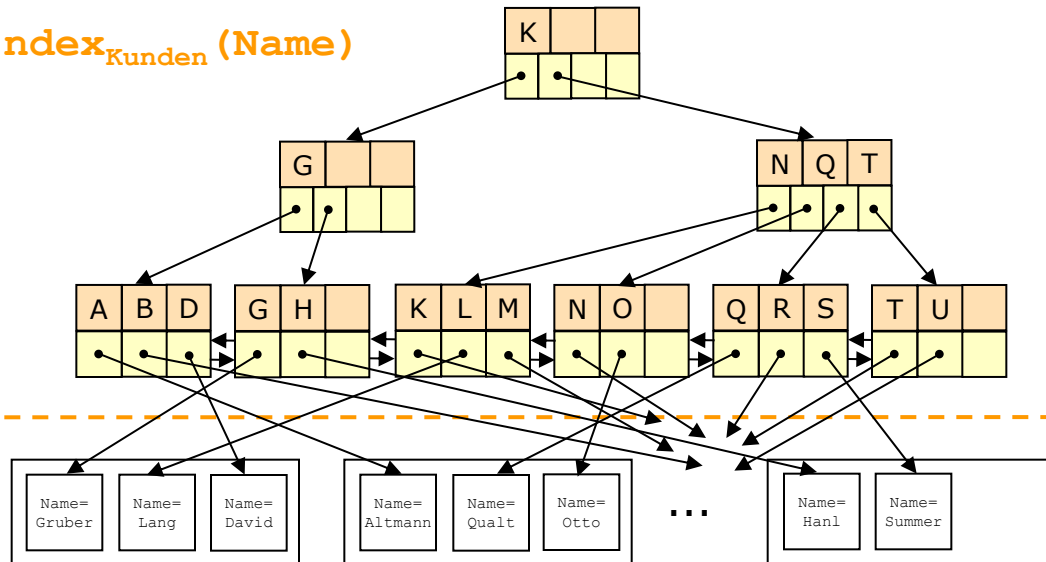
Indexstrukturen

Sekundärindex

- Datendatei
 - **nicht** nach Suchschlüssel geordnet
 - Suchschlüssel kann mehrfach vorkommen
- Indexdatei
 - sequentiell nach Suchschlüssel geordnet
- Effiziente Zugriffsarten:
 - sehr schnell für Punktanfragen
 - Mehrpunkt- und Bereichsanfragen: gut wenn nur kleiner Teil der Tabelle zurückgeliefert wird (wenige %)
 - besonders für nicht-sequentiellen Zugriff (random access) geeignet

Index_{Kunden} (Name)

Indexdatei



Mehrstufiger Index
(Dateiorganisation als
balancierter B⁺-Suchbaum)

Dicht besetzter Index

Keine Cluster

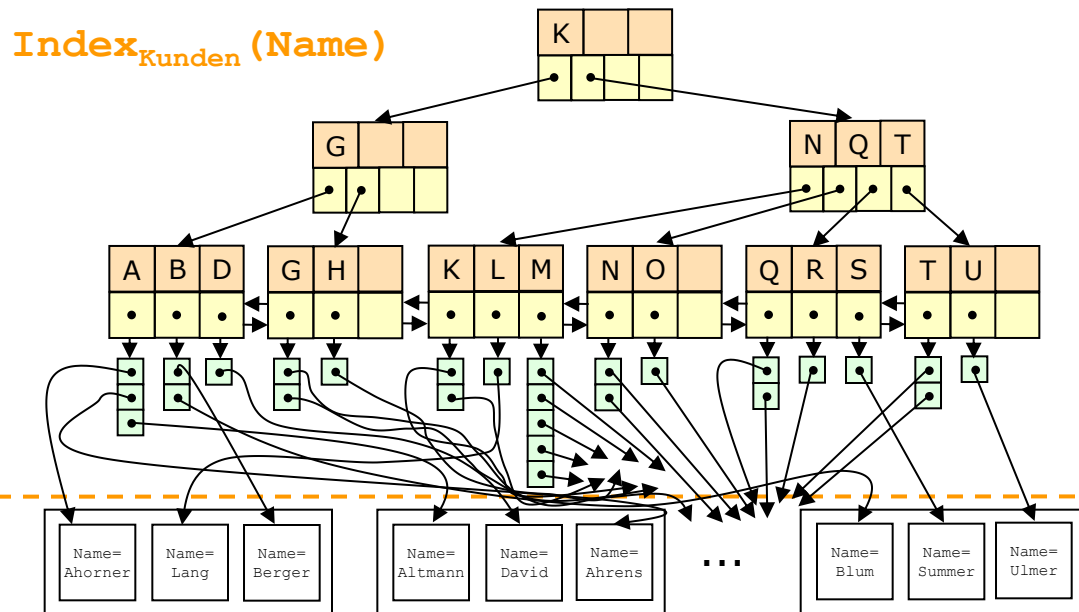
Datendatei

Indexstrukturen

Sekundärindex

■ Zwei Möglichkeiten für Zeigerabbildung

- **Mehrfache Indexeinträge**
 - ein Indexeintrag für jeden Datensatz
- **Buckets**
 - ein Indexeintrag für jeden Suchschlüssel
 - Indexeintrag zeigt auf ein Bucket
 - Bucket enthält Zeiger auf Datensätze



Indexdatei

Mehrstufiger Index
(Dateiorganisation als
B⁺-Baum)

Dicht besetzter Index

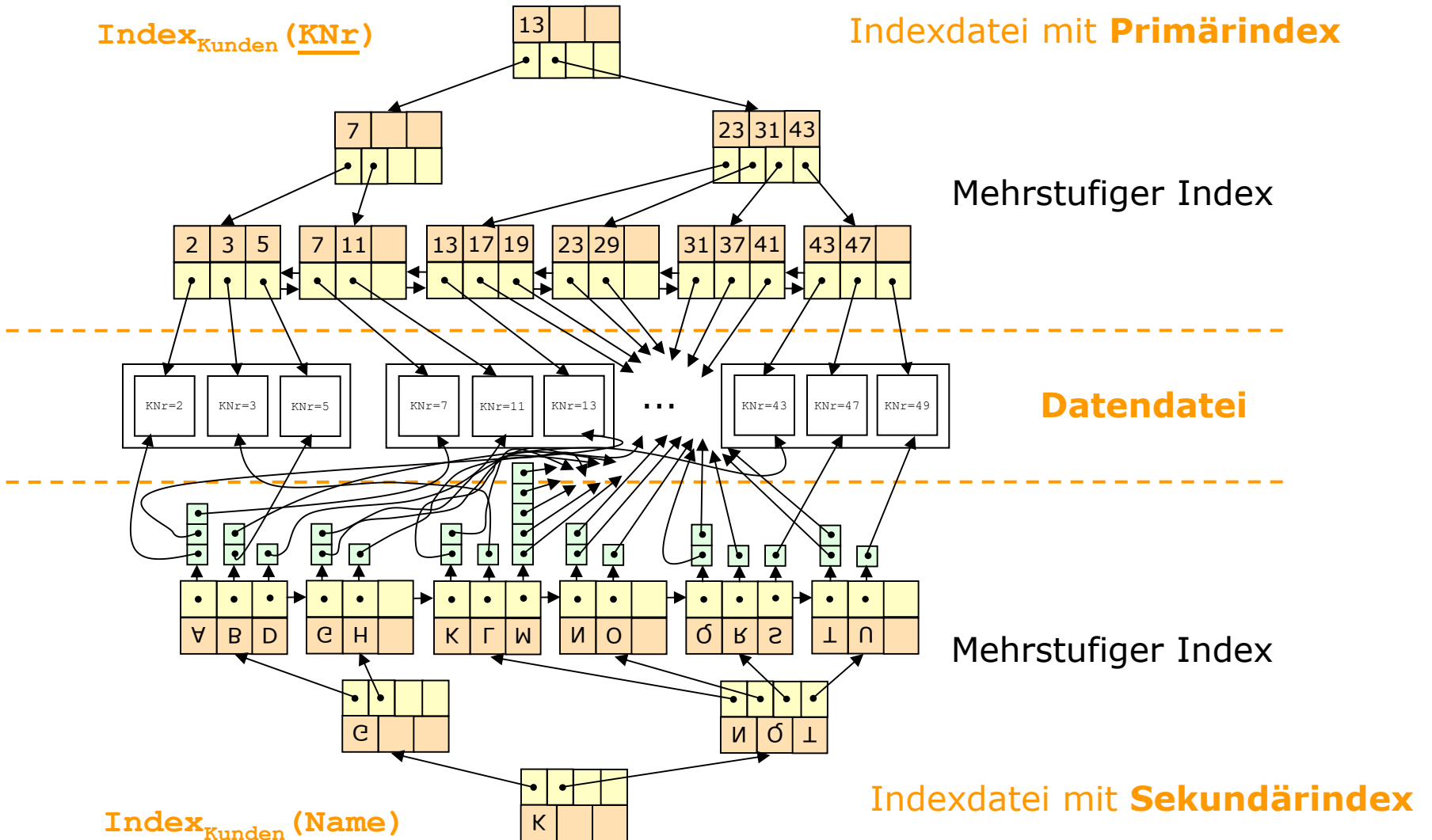
Buckets (Zeigerliste)

Keine Cluster

Datendatei

Indexstrukturen

Kombination Primär- und Sekundärindex



Indexstrukturen

Primär- versus Sekundärindex

- nur ein Primärindex (bzw. ein Clustered Index) pro Tabelle möglich
- mehrere Sekundärindizes pro Tabelle möglich
- Sekundärindex für schnellen Zugriff auf alle Felder, die nicht Suchschlüssel des Primärindex sind
- **Beispiel:** Tabelle **Kunde** mit Primärindex auf **KNr**
 - Finde alle Kunden mit **Ort=Linz**
 - Finde alle Kunden mit **Umsatz 100.000 bis 200.000**
- Ohne Index können diese Anfragen **nur durch sequentielles Lesen** aller Knoten beantwortet werden – sehr langsam
- **Sekundärindex** für schnellen Zugriff **erforderlich**

Indexstrukturen

Gegenüberstellung von Indextypen

- Alle Indextypen machen **Punktanfragen erheblich schneller.**
- Index erzeugt **Kosten bei Datenaktualisierung (DML-Operationen)**
--> Index muss ebenfalls aktualisiert werden.
- **Sortiert lesen** (=sequentielles Lesen nach Suchschlüsselordnung):
 - mit Primärindex schnell
 - mit Sekundärindex teuer, da sich aufeinander folgende Datensätze auf unterschiedlichen Seiten/Blöcken befinden (können)

Überblick

- Internes Schema
- Abbildung konzeptuelle auf interne Ebene
- Dateiorganisation und Zugriffspfade
- Indexierung
- Indexstrukturen in SQL (ORACLE)

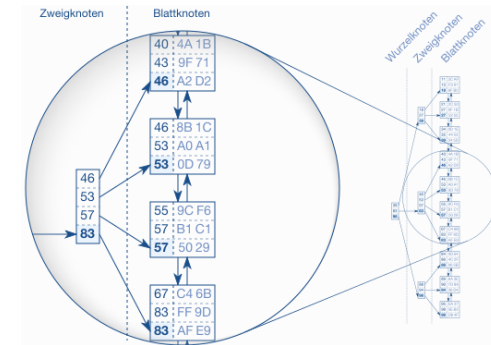
Indexstrukturen in SQL

- **SQL-Standard definiert keine Syntax** für Indizes
 - es hat sich aber eine gebräuchliche Syntax durchgesetzt
 - diese wird anhand von Oracle dargestellt
- **Datenbanksysteme realisieren spezifische Indizes:**
 - Indizes werden als eigenständige Schemaobjekte behandelt.
 - Indizes-Verwaltungsinformationen werden von Datenbanksystemen automatisch erstellt.
 - Indizes sind in der Regel als B⁺-Bäume realisiert
- **Vorteile:**
 - Beschleunigung des Zugriffs
 - Gewährleistung der Clustereigenschaft für Tabellen
- **Nachteile:**
 - erhöhter Verwaltungsaufwand
 - höherer Speicheraufwand (Indizes benötigen Speicherplatz)

Indizes 1/10

ORACLE

- Standardindex ist als **B⁺-Baum** aufgebaut.
- Index kann max. 32 Spalten umfassen.
- Tabellen sind uneingeschränkt mehrfach indizierbar.
- **Funktionsbasierte Indizes** sind auf den Ergebniswerten von Funktionsaufrufen über Spalten definiert.
- **Zusammengesetzte Indizes** umfassen mehrere Spalten.
- **Bitmap-Indizes** speichern Bitmatrizen für Aufzählungsattribute.
- **Covering Indizes** vermeiden Tabellenzugriffe.
- **Cluster-Indizes** erlauben Clusterung auch mehrerer Tabellen.
- Speicherorganisation erfolgt als **Heap-** oder **Index-**organisierte Tabelle.
- ...



Indizes 2/10

ORACLE - Eindeutige Indizes (Primärindex)

- Indizes werden **automatisch** für
 - **PRIMARY KEY**- und
 - **UNIQUE**-Constraintsangelegt.

CREATE INDEX-Anweisung

```
CREATE [UNIQUE] INDEX Indexname  
ON Tabelle (Attribut1[ASC, DESC], ..., Attributn[ASC, DESC]);
```

Implizite CREATE UNIQUE INDEX-Anweisung

```
CREATE TABLE Tabelle  
  (Attribut1 PRIMARY KEY USING INDEX  
    (CREATE UNIQUE INDEX Indexname ON  
      Tabelle(Attribut1)),  
  Attributn);
```

Indizes 3/10

ORACLE - Eindeutige Indizes (Primärindex)

CREATE TABLE-Anweisung

```
CREATE TABLE Employee (
  employee_id NUMBER NOT NULL,
  first_name VARCHAR2(1000) NOT NULL,
  last_name VARCHAR2(1000) NOT NULL,
  date_of_birth DATE NOT NULL,
  phone_number VARCHAR2(1000) NOT NULL,
  CONSTRAINT employee_pk PRIMARY KEY (employee_id)
);
```

```
SELECT *
FROM Employee
WHERE employee_id = 123;
```

Tabellenzugriff mittels ROWID

OPERATION	OBJECT_NAME	CARDINALITY	COST
SELECT STATEMENT			1
TABLE ACCESS (BY INDEX ROWID) EMPLOYEE		1	1
INDEX (UNIQUE SCAN) EMPLOYEE_PK	EMPLOYEE_PK	1	1
Access Predicates EMPLOYEE_ID=123			

Index wird verwendet!

Indizes 4/10

ORACLE - Zusammengesetzter Primärschlüssel

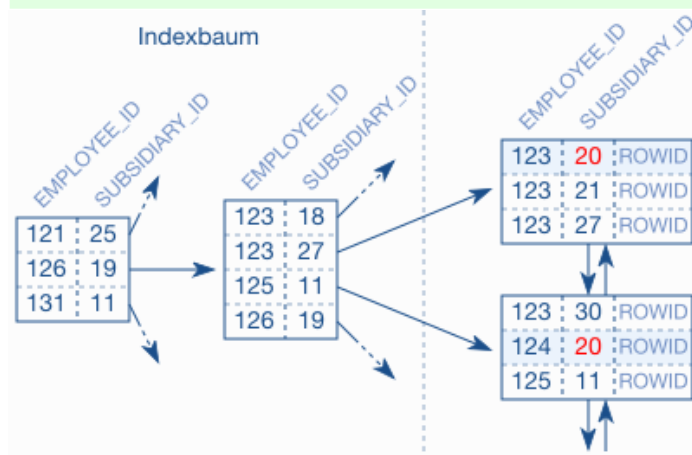
- Index umfasst alle Spalten des Primärschlüssels
- **Spaltenreihenfolge** im Index **ausschlaggebend!**

CREATE TABLE-Anweisung

```
CREATE TABLE Employee (
  employee_id NUMBER NOT NULL,
  subsidiary_id NUMBER NOT NULL,
  ...
  CONSTRAINT employee_pk PRIMARY KEY (employee_id, subsidiary_id));
```

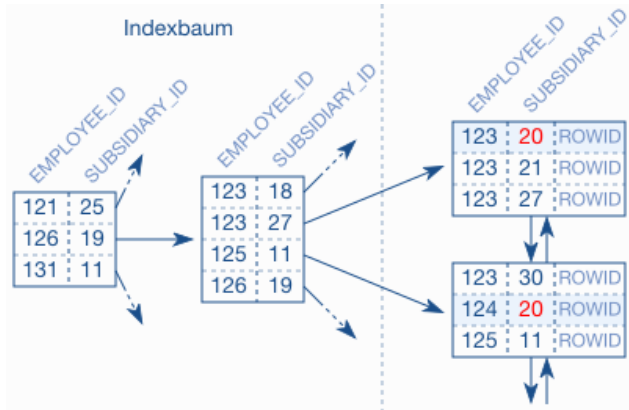
// Index wie folgt automatisch definiert

```
CREATE UNIQUE INDEX employee_pk ON employee (employee_id, subsidiary_id);
```



Indizes 5/10

ORACLE - Zusammengesetzter Primärschlüssel



```
SELECT *
FROM Employee
WHERE employee_id = 123
AND subsidiary_id = 30;
```

OPERATION	OBJECT_NAME	CARDINALITY	COST
SELECT STATEMENT			1
TABLE ACCESS (BY INDEX ROWID)	EMPLOYEES	1	1
INDEX (UNIQUE SCAN)	EMPLOYEES_PK	1	1
Access Predicates			
AND			
	EMPLOYEE_ID=123		
	SUBSIDIARY_ID=30		

Index wird verwendet!

```
SELECT *
FROM Employee
WHERE subsidiary_id = 20;
```

OPERATION	OBJECT_NAME	CARDINALITY	COST
SELECT STATEMENT			2
TABLE ACCESS (FULL)	EMPLOYEES	1	2
Filter Predicates			
	SUBSIDIARY_ID=20		

Index wird nicht verwendet!

```
CREATE UNIQUE INDEX employee_pk ON employee (subsidiary_id, employee_id);
```

Indizes 6/10

ORACLE - Sekundärindizes

```
SELECT first_name, last_name, subsidiary_id, phone_number
FROM Employee
WHERE last_name = 'WINAND';
```

```
CREATE INDEX emp_name ON employee (last_name);
```

Id	Operation	Name	Rows	Cost
0	SELECT STATEMENT		1	3
* 1	TABLE ACCESS BY INDEX ROWID	EMPLOYEES	1	3
* 2	INDEX RANGE SCAN	EMP_NAME	1	1

Predicate Information (identified by operation id):

1 - filter("SUBSIDIARY_ID"=30)
2 - access("LAST_NAME"='WINAND')

Niedriger Cost-Wert, da Index
nur wenige Zeile liefert!

■ Ausführungsplan zeigt

- INDEX RANGE SCAN liefert nur eine Zeile (wenn nur ein Mitarbeiter mit LAST_NAME = 'WINAND' vorhanden ist)
- TABLE ACCESS BY INDEX ROWID muss daher nur eine Zeile aus der Tabelle laden

Indizes 7/10

ORACLE - Funktionsbasierte Indizes

- Wird in der **WHERE-Klausel** ein **Ausdruck** oder eine **Funktion** verwendet, so kann dabei **kein Standardindex** genutzt werden.

```
SELECT first_name, last_name, phone_number
FROM Employee
WHERE UPPER(last_name) = UPPER('winand');
```

Id	Operation	Name	Rows	Cost
0	SELECT STATEMENT		10	477
* 1	TABLE ACCESS FULL	EMPLOYEES	10	477

Predicate Information (identified by operation id):

1 - filter(UPPER("LAST_NAME")='WINAND')

TABLE ACCESS FULL, obwohl ein Index auf LAST_NAME existiert!

UPPER-Funktion wird als BLACKBOX () betrachtet!

Funktionsbasierter Index

```
CREATE INDEX emp_up_name ON Employee (UPPER(last_name));
```

Achtung: UPPER und LOWER werden von ORM-Tools manchmal ohne Wissen der Entwickler verwendet. Hibernate verwendet zum Beispiel die LOWER-Funktion, um Case-insensitive Suchen umzusetzen.

Indizes 8/10

ORACLE - Zusammengesetzte Indizes

- Kandidaten: **Spaltenkombinationen**, die vorwiegend gemeinsam in `WHERE`-Klauseln vorkommen (`AND`-Verknüpfung)
- Spalten, die auch in Einzelsuchkriterien verwendet werden, sollten als **führende Komponenten** des Index definiert werden (Indiziere zuerst für die Ist-gleich-Bedingungen, dann für andere)

Index auf Spalten x , y , z der Tabelle t bei häufigen Abfragen auf die Kombination (y, z) :

```
CREATE INDEX comp_index ON t(y,z,x);
```

Indizes 9/10

ORACLE - Zusammengesetzte Indizes

CREATE TABLE/INDEX-Anweisung

```
CREATE TABLE Employee (  
  employee_id      NUMBER          NOT NULL,  
  first_name       VARCHAR2(1000)  NOT NULL,  
  last_name        VARCHAR2(1000)  NOT NULL,  
  date_of_birth    DATE            NOT NULL  
    CONSTRAINT employee_pk PRIMARY KEY (employee_id)  
);  
CREATE INDEX emp_comp_index ON Employee (last_name, first_name, date_of_birth);
```

emp_comp_index wird verwendet

```
SELECT * FROM Employee WHERE last_name = 'WINAND';  
  
SELECT * FROM Employee WHERE last_name = 'WINAND' AND first_name = 'JOE';  
  
SELECT * FROM Employee WHERE last_name = 'WINAND'  
                             AND first_name = 'JOE'  
                             AND date_of_birth = '1966-08-09';
```

emp_comp_index wird nicht verwendet

```
SELECT * FROM Employee WHERE first_name = 'JOE';  
  
SELECT * FROM Employee WHERE date_of_birth = '1966-08-09';  
  
SELECT * FROM Employee WHERE last_name = 'WINAND' -- emp_comp_index für last_name  
                             AND date_of_birth = '1966-08-09'; -- scan notwendig
```

Indizes 10/10

ORACLE - Indexverwaltung

Indexverwaltung

```
DROP INDEX Indexname;  
DROP INDEX Indexname RENAME TO NeuerIndexname;  
  
ALTER INDEX Indexname UNUSABLE;  
ALTER INDEX Indexname REBUILD [ONLINE];  
  
ALTER INDEX Indexname MONITORING USAGE;  
ALTER INDEX Indexname NOMONITORING USAGE;  
  
...
```

Indexnutzung

```
SELECT index_name, table_name, used, start_monitoring, ...  
FROM v$object_usage;
```

Index

Wann soll ein Index verwendet werden?

■ Kandidaten für Indizierung

- Primärschlüssel und `UNIQUE`-Spalten (automatisch!)
- Fremdschlüssel-Spalten (meistens, vor allem bei Verbund!)
- Spalten, die häufig in `WHERE`-Klauseln von signifikanten, selektiven Anweisungen verwendet werden
- Spalten, die in der `ORDER BY`-Klausel oder von Funktionen wie `MAX` oder `MIN` verwendet werden
- Spalten enthalten großen Wertebereich oder viele `NULL`-Werte
- Tabelle wird selten aktualisiert

■ Vorsicht bei

- Hoher `INSERT/UPDATE/DELETE`-Frequenz auf indizierte Attribute, da Index **reorganisiert** werden muss
- Basistabellen mit wenigen Datensätzen (Index ab ~ 200 Zeilen)

Index

Wann wird ein Index vom Anfrageoptimierer verwendet?

■ Kardinalität

= Anzahl der Werte einer Spalte mit unterschiedlichen Ausprägungen

CUSTOMER

Customer_ID	Business_Size
1	S
2	L
3	M
4	M
5	S

■ Selektivität

= Verhältnis Kardinalität einer indizierten Spalte zur Anzahl der Datensätze

Indexerstellung

```
CREATE INDEX Customer_Business_Size
ON Customer(Business_Size);
```

Kardinalität = 3 (S, M, L)

Selektivität = 0.6 (3/5) [bei 10.000 Customer = 0.0003]

Berechnung Selektivität in Oracle

```
SELECT (distinct_keys / num_rows) AS Selectivity
FROM dba_indexes
WHERE index_name LIKE UPPER('Customer_Business_Size');
```



Ergebnis

SELECTIVITY
0.6

Berechnung Selektivität in Oracle – Index Primary Key

```
SELECT (distinct_keys / num_rows) AS Selectivity
FROM dba_indexes
WHERE index_name LIKE UPPER('SYS_C0017971');
```



Ergebnis

SELECTIVITY
1.0

Faustregel: Index nur bei hoher Selektivität, d.h. bei kleiner Treffermenge

Zusammenfassung

- Indizes **beschleunigen** Punkt-, Bereichs- und Verbundanfragen erheblich.
- Indizes **verlangsamen** das Einfügen, Löschen und Ändern von Tupeln der Relation.
 - Indizes müssen **automatisch nachgeführt** werden.
- Indizes benötigen **Speicherplatz** und **aktuelle Statistiken**
- **Indexerzeugung** kann lange dauern.
 - **Bulk Load**: Es ist (viel) effizienter, zuerst die Daten in die Tabelle einzufügen und nachher alle Indizes zu erstellen als umgekehrt.
- Indizesdefinition ist eine der schwierigsten Aufgaben des Datenbankdesigns
 - Vorhersage der **Query Workload** und **Änderungsfrequenz**
 - **Attributwahl**
 - häufiger Vergleich mit Konstanten
 - häufiges Verbundprädikat